Lab 07 — False Positives & Surgical Tuning

> **Level:** Advanced **Goal:** Diagnose a false positive from the audit log and fix it **without** weakening protection anywhere else — the single most important production skill for a WAF operator. **Time:** ~50 minutes **Operator focus:** This is what WAF engineers actually spend most of their time doing. Blocking attacks is easy; *not* blocking your own customers while still blocking attacks is the hard, valuable part.

---

1. Theory — why false positives are the real problem

A WAF matches *patterns*, and legitimate traffic sometimes looks like an attack. A product search for **"Compare and select from our items"** contains the SQL-ish sequence *"select … from"*, so CRS's libinjection rule flags it.

Two ways to get this wrong:

- **Do nothing** → real users get `403`s on innocent input. The business asks you to "turn the WAF off." Everyone loses.
- **Over-correct** → you run `SecRuleRemoveById 942100` and disable SQLi detection globally. Now the site is wide open. You traded a support ticket for a breach.

The professional answer is a **surgical exclusion**: turn off *exactly one rule*, for *exactly one parameter*, on *exactly one path* — and nowhere else.

> **Why this matters:** "Security vs usability" is a false choice if you tune precisely. The whole craft is scoping your exclusions as narrowly as the false positive requires — not one character wider.

---

2. Reproduce the false positive

Against the normal stack (`:8080`):

```
curl -s -o /dev/null -w "%{http_code}\n" \
  --get "http://localhost:8080/search" \
  --data-urlencode "q=Compare and select from our items"
```

Returns `403`. A real customer just got blocked.

---

3. The operator workflow — always start from the log

Never guess which rule to exclude. Read it out of the audit log:

```
docker logs modseclabs-waf | tail -n 40 | \
  grep -o '"ruleId":"[0-9]*"[^}]*"data":"[^"]*"'
```

You'll see the culprit and *exactly* what it matched:

```
"ruleId":"942100", ... "data":"Matched Data: n&Ekn found within ARGS:q: Compare and select fr
```

Three facts to extract, every time:

Fact	Value here	Used for
Rule ID	`942100`	which rule to scope
Variable (target)	`ARGS:q`	which parameter to scope
URI	`/search`	which path to scope

That's your exclusion, fully specified: *rule 942100, arg q, path /search*.

> ⚠️ **Read this or you'll tune the wrong rule.** A block always produces **two** kinds of log line:
>
> - the **gate** — rule `949110`, *"Inbound Anomaly Score Exceeded (Total Score: 5)"*. This fires on **every** block and is **never** what you exclude. Removing `949110` would switch off blocking for the whole site.
> - the **cause** — the rule that actually matched your input and *added* the score. It's the line whose `data` says *"Matched Data … within `ARGS:q`"*. **That** is the ID you scope.
>
> Also: **the cause's ID depends on your CRS version.** On the CRS 4.28 used here it's `942100` (libinjection). On another build the same phrase may be caught by `942150` / `942260` /etc. **Always take the ID from *your* log's "Matched Data" line — never copy `942100` blindly from this sheet.**

Here the whole block came from a **single** scoring rule: `942100` added +5, and the threshold is 5. So excluding just that one rule drops the score to **0** and the request passes cleanly — the anomaly gate never triggers. (When an attack trips several rules at once, one exclusion won't unblock it — and that's exactly what you want.)

---

4. Choosing the right tool — from blunt to surgical

ModSecurity/CRS give you a ladder of exclusion mechanisms. Prefer the most specific one that fixes the problem:

Mechanism	Scope	Use when
`SecRuleRemoveById 942100`	Rule off **everywhere**	Almost never. Last resort.
`SecRuleUpdateTargetById 942100 "!ARGS:q"`	Rule ignores arg `q` on **every** path	The arg is safe app-wide.

Mechanism	Scope	Use when
`ctl:ruleRemoveTargetById=942100;ARGS:q` **inside a path condition**	Rule ignores arg `q` **only on** `/search`	✅ The surgical fix — our choice.
`ctl:ruleRemoveById=942100` inside a path condition	Whole rule off, but only on one path	The whole rule is wrong for that endpoint.

> **Deep-dive —** `SecRuleRemove*` **vs** `ctl:` **:** `SecRuleRemove*` directives are evaluated at **config-load time** and are unconditional. The `ctl:` action runs **per-request**, so you can wrap it in a `SecRule` that checks the URI — giving you path-scoped exclusions. That conditional power is why CRS's own exclusion files use `ctl:` .

---

5. Write the surgical exclusion

Exclusions must run **before** the CRS rule they modify, so they go in a `REQUEST-900-*` file (loads before the `9xx` detection rules). Create `rules/EXCLUSIONS-BEFORE-CRS.conf` :

```
# Only on /search, tell rule 942100 to ignore the `q` argument.
# Everything else keeps full SQLi protection.
SecRule REQUEST_URI "@beginsWith /search" \
    "id:1900001,phase:1,pass,nolog,\
     ctl:ruleRemoveTargetById=942100;ARGS:q"
```

Load it and start a tuned WAF:

```
docker run -d --name modseclabs-waf-tuned \
  --add-host=host.docker.internal:host-gateway \
  -e BACKEND="http://host.docker.internal:5000" -e PORT=8080 \
  -e MODSEC_RULE_ENGINE=On -e PARANOIA=1 -e ANOMALY_INBOUND=5 \
  -e MODSEC_AUDIT_LOG=/dev/stdout -e MODSEC_AUDIT_ENGINE=RelevantOnly \
  -v "$PWD/rules/EXCLUSIONS-BEFORE-CRS.conf:/etc/modsecurity.d/owasp-crs/rules/REQUEST-900-LO
  -p 8092:8080 owasp/modsecurity-crs:nginx
```

---

6. Verify — the exclusion must be *tight*

The whole point is that you fixed the FP **and nothing else**. Test all three:

```
# 1. The false positive is gone:
curl -s -o /dev/null -w "benign search        -> %{http_code} (want 200)\n" \
  --get "http://localhost:8092/search" --data-urlencode "q=Compare and select from our items"

# 2. A REAL SQLi on the SAME parameter is STILL blocked:
curl -s -o /dev/null -w "real SQLi on q        -> %{http_code} (want 403)\n" \
  --get "http://localhost:8092/search" --data-urlencode "q=1' UNION SELECT password FROM user
```

```
# 3. SQLi on a DIFFERENT parameter is untouched:
curl -s -o /dev/null -w "SQLi on /login user   -> %{http_code} (want 403)\n" \
  --get "http://localhost:8092/login" --data-urlencode "user=admin' OR '1'='1"
```

Result:

```
benign search        -> 200 (want 200)
real SQLi on q        -> 403 (want 403)
SQLi on /login user   -> 403 (want 403)
```

> **Why the real SQLi on `q` is still blocked:** we only excluded rule `942100`. The `UNION SELECT … FROM` payload also trips **other** `942xxx` rules (`942190`, `942360`, …), so the anomaly score still crosses the threshold. **This is the payoff of scoring + narrow exclusions:** removing one rule for one arg barely dents real-attack coverage, because attacks trip many rules. That is the entire argument for surgical tuning.

---

7. The real-world go-live workflow

Putting Labs 02, 05 and 07 together, here is how a WAF actually gets deployed in industry:

1. **Deploy in `DetectionOnly` at PL1.** Block nothing yet.
2. **Collect logs** for days/weeks of real traffic (or replay prod traffic).
3. **Triage every alert:** for each, read `ruleId` + `data` and decide *true positive* (an attack — good) or *false positive* (tune it).
4. **Write narrow exclusions** for the false positives, version-controlled in a `REQUEST-900-*` file. Re-test that attacks are still caught.
5. **Flip to `SecRuleEngine On`** only when the false-positive rate is near zero.
6. **Iterate forever:** new app features create new false positives; new CVEs create new rules. A WAF is a *maintained* system, not a set-and-forget box.

> **CRS also ships automatic exclusion packages** for popular apps (WordPress, Drupal, etc.) via `SecAction … setvar:tx.crs_exclusions_wordpress=1`. When you run a known app, enable its package before hand-writing exclusions — someone already did the tuning for you.

Clean up

```
docker rm -f modseclabs-waf-tuned
```

---

8. Recap

- False positives, not attacks, are the operator's main day-to-day problem.

- **Always diagnose from the audit log:** extract rule ID, target arg, URI.
- Prefer the **narrowest** exclusion: path + rule + arg, using `ctl:ruleRemoveTargetById` inside a URI condition.
- Verify the fix is tight — FP gone, real attacks (even on the same param) still blocked.
- Real deployments follow **DetectionOnly → tune → On**, forever.

Next: Lab 08 — the finale. Virtual-patch a real CVE (Log4Shell), then a hard, honest look at **where ModSecurity fails** and why a WAF is never your only defence.